Phase 2: Technical Design & System Requirements

Graduation Project: Geo-Bound Crypto-Vault with Automated DevSecOps Pipeline

Domain: Encrypted Vault Design & Key Management Architecture
Assigned Role: Student 2
Tech Stack: Python, AES-256-GCM, PBKDF2 / Argon2, SQLite
Submission Date: Today

---

1. Encrypted Vault System Requirements

A. Functional Requirements (FR)

- **File & Data Ingestion:** The system allows authorized users to upload sensitive files and text payloads for encryption and secure storage.
- **Automated In-Memory Encryption:** Files must be encrypted dynamically in memory (RAM) before being written to disk or the database.
- **Decryption & Retrieval Management:** Authorized users can request decryption and retrieve original files upon successful authentication and policy checks.
- **Secure Deletion:** Secure purging of encrypted payloads and metadata from the database without leaving temporary unencrypted artifacts.

B. Non-Functional & Security Requirements (NFR)

- **Confidentiality (End-to-End Encryption):** Complete payload secrecy ensuring that raw data is unreadable, even to Database Administrators (DBAs).
- **Integrity (Authenticated Encryption):** Use of authenticated encryption schemes to instantly detect data tampering or corruption in the database.
- **Performance & Efficiency:** Optimized crypto-operations ensuring minimal latency and memory overhead during file payload transformations.

2. Encryption Scheme & Key Management Architecture

Addressing advisor feedback regarding key security, the strength of AES-256 is paired with a robust zero-trust key management system:

> **Encryption Algorithm: AES-256-GCM (Galois/Counter Mode)**
> AES-256-GCM was selected because it provides both high-speed confidentiality and built-in authenticity tags (AEAD), guaranteeing immediate detection of ciphertext modification.

Key Management Strategy:

1. **Zero Plaintext Storage:** Master encryption keys and decryption payloads are never stored in plaintext within SQLite or configuration files.
2. **Dynamic Key Derivation (PBKDF2 / Argon2):** Encryption keys are dynamically derived from user credentials combined with unique cryptographic salts using over 100,000 iterations to thwart dictionary and rainbow table attacks.
3. **Per-File Initialization Vectors (IV / Nonce):** Every encrypted file generates a unique 96-bit random IV to ensure identical files yield completely different ciphertexts.

4. **Volatile Memory Lifecycle:** Derived keys exist only in volatile RAM for the duration of the active cryptographic operation and are immediately scrubbed upon session termination.

3. Database Schema Structure (SQLite)

A clean, lightweight **SQLite** database schema is designed to persist user metadata and encrypted payloads securely without storing sensitive unencrypted information:

A. Users Table (`users`)

Column Name	Data Type	Description & Constraints
`user_id`	INTEGER	Primary Key, Auto-increment — Unique user identifier
`username`	TEXT	Unique, Not Null — Account login identifier
`password_hash`	TEXT	Not Null — Argon2id/Bcrypt hash for user authentication
`user_salt`	BLOB	Not Null — Cryptographic salt used for key derivation
`created_at`	TIMESTAMP	Timestamp of user registration

B. Encrypted Vault Files Table (`vault_files`)

Column Name	Data Type	Description & Constraints
`file_id`	TEXT (UUID)	Primary Key — Unique file identifier
`user_id`	INTEGER	Foreign Key (Users) — Owner account mapping
`file_name_encrypted`	TEXT	Encrypted original filename for privacy protection
`encrypted_data`	BLOB	Not Null — Encrypted file binary payload (Ciphertext)
`encryption_iv`	BLOB	Not Null — Random Initialization Vector (12-byte Nonce)
`auth_tag`	BLOB	Not Null — AES-GCM Integrity Authentication Tag
`file_size_bytes`	INTEGER	Size of encrypted payload in bytes
`created_at`	TIMESTAMP	Upload and encryption timestamp

4. Vault Operations Workflow

Encryption & File Ingestion Workflow:

1. User submits a file via the frontend interface and provides their passphrase.
2. The backend derives the temporary `AES-Key` via `PBKDF2(Password, User_Salt)`.
3. A cryptographically secure random 12-byte `IV` is generated.
4. The payload is encrypted via `AES-256-GCM`, returning the `Ciphertext` and `Auth Tag`.
5. Only the encrypted binary parameters are written to SQLite; cleartext memory is purged.

Decryption & Retrieval Workflow:

1. User requests file retrieval and passes all security gate checks (Geo-fence & Attestation).
2. System retrieves `encrypted_data`, `encryption_iv`, and `auth_tag` from SQLite.
3. The backend reconstructs the encryption key using the user's passphrase and stored `user_salt`.
4. AES-GCM verifies the integrity using the `auth_tag`. If altered, execution aborts instantly.
5. Upon successful authentication, the decrypted file is streamed back to the client.